

READ: REINFORCEMENT-GUIDED CAPABILITY DISTILLATION FOR LARGE LANGUAGE MODELS

Anonymous authors

Paper under double-blind review

ABSTRACT

Capability distillation applies knowledge distillation to selected model capabilities, aiming to compress a large language model (LLM) into a smaller one while preserving the abilities needed for a downstream task. However, most existing methods treat capabilities as independent training targets and overlook how improving one capability can reshape the student’s broader capability profile, especially when multiple abilities jointly determine task success. We study capability distillation under a fixed token budget and identify two consistent patterns: distillation induces systematic, budget-dependent cross-capability transfer, and additional budget often brings limited task-relevant gains while sometimes degrading other useful abilities. Building on these insights, we propose *ReAD*, a Reinforcement-guided cApability Distillation framework that explicitly accounts for capability interdependence. ReAD first infers task-essential capabilities, then generates capability-targeted supervision on the fly, and finally uses an uncertainty-aware contextual bandit to adaptively allocate the distillation budget based on expected utility gains. Extensive experiments show that ReAD improves downstream utility under the same token budget while reducing harmful spillover and wasted distillation effort compared to strong baselines. Our code is available at <https://anonymous.4open.science/r/ReAD-62B7>.

1 INTRODUCTION

Knowledge distillation (KD) (Hinton et al., 2015) for large language models (LLMs) (Zhao et al., 2023) has become an essential research direction for enhancing the accessibility and efficiency of Machine-Learning-as-a-Service (MLaaS) (Cai et al., 2024; Xu et al., 2024; Yang et al., 2024): **a small LLM learns to imitate a large teacher’s outputs and can then be deployed under limited computational or financial resources** (Sanh et al., 2019; Jiao et al., 2020; Wang et al., 2020; Li et al., 2021; Liang et al., 2023). Recent studies show that capability distillation, a specialization of knowledge distillation that focuses supervision on a target capability (e.g., instruction-following, reasoning, mathematics, or coding), can substantially improve the student’s performance on downstream tasks that primarily depend on that capability, while achieving these gains at lower serving cost than the large teacher model (Magister et al., 2023; Shridhar et al., 2023; Xu et al., 2023a; Yue et al., 2024).

Despite rapid progress in capability distillation, most existing methods still treat capabilities as if they can be improved independently (Taori et al., 2023; Chiang et al., 2023; Zhang et al., 2024), even though empirical evidence suggests that optimizing one capability often triggers broad, unintended shifts in the model’s overall capability profile (Zhong et al., 2024; Fang et al., 2025; Cloud et al., 2025). **This mismatch matters most under a fixed token budget, where downstream utility depends on the targeted capability and on the other capabilities the task implicitly requires. Ignoring cross-capability interactions then wastes tokens in two ways: the extra gain on the target shrinks as its budget grows, and the same updates can degrade non-target capabilities the task needs. We call tokens spent on such updates *budget waste*. To examine this gap, we run a controlled empirical study: we repeatedly distill the student under several budgets, each time spending the entire budget on one of eight core capabilities (General Knowledge, Reasoning, Math, Code, Tool Use, Long-Context Understanding or LCU, Steerability, and Multilinguality), and evaluate it on all of them. This yields a budget-dependent capability transfer matrix whose diagonal holds on-target gains and whose off-diagonal entries show how performance is redistributed.** Across budgets, we observe two consistent patterns: (i) *capability-specific distillation induces systematic, budget-dependent transfer to other*

capabilities rather than isolated improvements, and (ii) increasing the budget for a single target produces smaller additional target gains while making the average harm to non-target capabilities more pronounced, which together explain why naively scaling the budget for one capability can be inefficient and why the allocation itself should be optimized.

Building on these observed insights, we propose *ReAD*, a Reinforcement-guided cApability Distillation framework for large language models (Figure 2). *ReAD* first infers a task requirement vector that identifies the capabilities essential to downstream utility and treats degradations on them as harmful spillover. It then spends the budget in short steps: at each step it samples capability-labeled prompts according to the current allocation, queries the teacher, distills the student on the answers with a standard loss, and feeds the resulting capability-profile change to an uncertainty-aware contextual bandit, whose reward favors task-aligned gains and penalizes spillover and token cost. Experiments show that *ReAD* strengthens task-relevant capabilities and wastes less budget than one-hot distillation, static and greedy allocation schedules, and capability-specific distillation methods, at both the 20M and the 150M token budgets.

To sum up, this paper makes the following key contributions:

- **Empirical study of cross-capability interactions.** Distilling toward a single capability systematically changes the others, an interaction that existing methods ignore.
- **Budgeted capability distillation formulation.** We cast capability distillation as allocating a fixed budget across interacting capabilities, trading utility against side effects.
- **ReAD: reinforcement-guided capability distillation.** *ReAD* infers task-relevant capabilities, generates targeted data with controlled difficulty, and allocates the budget with an uncertainty-aware contextual bandit that adapts as the student changes.
- **Theory analysis of the capability interdependence.** A local analysis explains cross-capability transfer and diminishing returns, and why uncertainty-aware allocation helps.

2 PRELIMINARY

2.1 NOTATION AND FORMALIZATION

In this paper, we study budgeted capability distillation from a large teacher model T to a smaller student model S , with total training-token budget B . Let $\mathcal{C} = \{c_1, \dots, c_{|\mathcal{C}|}\}$ denote a set of measurable capabilities, here the eight core capabilities of Appendix Table 3, and let $s_k(M)$ be model M 's benchmark score on capability c_k . A distillation strategy π specifies how data is generated, how the teacher is queried, and how tokens are allocated across capabilities, assigning budgets $\{b_k\}$ with $\sum_k b_k \leq B$, and producing a distilled student S_π .

Definition 2.1 (Capability distillation). Given a teacher model T , an initial student model S_0 , and capability-specific data distributions $\{\mathcal{D}_c\}_{c \in \mathcal{C}}$, capability distillation allocates a limited budget B across capabilities to improve task-relevant performance under shared representations by optimizing a weighted mixture of distillation objectives:

$$\min_{S, \mathbf{w}} \mathbb{E}_{x \sim \mathcal{D}_{\mathbf{w}}} [\ell(S(x), T(x))] \quad \text{s.t.} \quad \mathbf{w} \in \Delta_{|\mathcal{C}|}, \text{cost}(S; S_0) \leq B. \quad (1)$$

where $\mathcal{D}_{\mathbf{w}} := \sum_{c \in \mathcal{C}} w_c \mathcal{D}_c$, and $\text{cost}(S; S_0)$ measures the distillation token budget consumed to obtain S from S_0 . Here $\mathbf{w} = (w_c)_{c \in \mathcal{C}}$ lies in the probability simplex $\Delta_{|\mathcal{C}|} = \{\mathbf{w} \in \mathbb{R}^{|\mathcal{C}|} \mid w_c \geq 0, \sum_{c \in \mathcal{C}} w_c = 1\}$, which specifies the allocation of training effort across capabilities, and ℓ denotes a standard distillation loss, such as token-level cross-entropy or logit-matching between the student's and the teacher's output distributions.

Under shared representations, budget spent on one capability changes the others. We measure this cross-capability effect by distilling with one-hot allocations:

Definition 2.2 (Capability interdependence). Let S_0 be an initial student model, and let $S(\mathbf{w}, B)$ denote the student obtained after capability distillation under allocation $\mathbf{w} \in \Delta_{|\mathcal{C}|}$ and budget B . For each capability c_i , define the one-hot allocation vector $\mathbf{w}^{(i)} \in \Delta_{|\mathcal{C}|}$ by $w_i^{(i)} = 1$ and $w_j^{(i)} = 0$ for all $j \neq i$. The *capability interdependence* under capability distillation is characterized by the capability



Figure 1: Exploratory study. (a–b) Cross-capability transfer under single-capability distillation: larger budgets sharpen target gains but expose stronger negative transfer. (c–d) Budget waste: extra tokens yield smaller target gains while increasing harmful spillover to non-target capabilities.

transfer matrix

$$\mathbf{T}_{ij}(B) := s_j(S(\mathbf{w}^{(i)}, B)) - s_j(S_0),$$

where $\mathbf{T}(B) \in \mathbb{R}^{|C| \times |C|}$, $\mathbf{T}_{ij}(B)$ quantifies the effect of distilling toward capability c_i on the performance of capability c_j . Non-zero off-diagonal entries indicate interdependence between capabilities under shared representations.

2.2 EXPLORATORY STUDY AND PROBLEM FORMULATION

We distill Llama-3.3-70B-Instruct into Llama-3.1-8B-Instruct under fixed token budgets, one target capability at a time, and evaluate the student on all benchmarks to form the transfer matrix of Definition 2.2 (strategies and settings in Appendix B). All benchmark metrics are accuracies or success rates on a 0–100 scale, a capability score is the unweighted mean over its benchmarks, and every entry of the transfer matrix is an absolute difference in these points (Definition 2.2); no further rescaling is applied (Appendix B).

Observation 1: Distilling a specific capability redistributes performance across other capabilities in a budget-dependent manner. Figure 1(a–b) shows the transfer matrices at a small and a large budget. The off-diagonal mass is consistently non-zero, so optimizing one capability moves the others rather than leaving them unchanged, and the structure is budget-dependent: at small budgets the changes are weak and diffuse, whereas at large budgets the diagonal sharpens and negative off-diagonals appear for specific non-targets.

Observation 2: Capability distillation exhibits substantial budget waste due to diminishing returns and negative spillover. For the strongest-improving targets, the marginal target gain of 20M→80M is consistently larger than that of 80M→150M (*diminishing returns*, Figure 1c), while the drop on non-target capabilities grows with the target’s budget (*negative spillover*, Figure 1d). Beyond a capability-specific knee point, extra tokens thus buy little target benefit and amplify cross-capability degradation. Neither effect is new in isolation: off-target drops are a form of catastrophic forgetting (Kirkpatrick et al., 2017; Luo et al., 2023), and saturation is ordinary diminishing returns. What allocation can exploit is their *structure*: transfer is signed, source-specific, and budget-dependent, so a token’s marginal value depends on where the budget has already gone.

Effective distillation under a fixed budget must therefore reason jointly about task-relevant capabilities, their interactions, and allocation. We formalize this as a sequential budget allocation problem across interdependent capabilities, as follows:

Problem 2.3 (Budget allocation for interdependent capability distillation). *Given a teacher model T , an initial student model S_0 , a downstream task τ , and a total distillation budget B , consider a sequential distillation process over steps $t = 1, \dots, T$ with cumulative cost not exceeding B . At each step t , a capability allocation vector $\mathbf{w}_t \in \Delta_{|C|}$ is selected and applied to update the student, yielding S_t . Let $U_\tau(\cdot)$ be the task-dependent utility function; our target is to determine an allocation policy $\{\mathbf{w}_t\}_{t=1}^T$ that maximizes the performance of the final student model:*

$$\max_{\{\mathbf{w}_t\}_{t=1}^T} U_\tau(S_T), \text{ s.t. } \sum_{t=1}^T \text{cost}(S_{t-1} \rightarrow S_t) \leq B \quad (2)$$

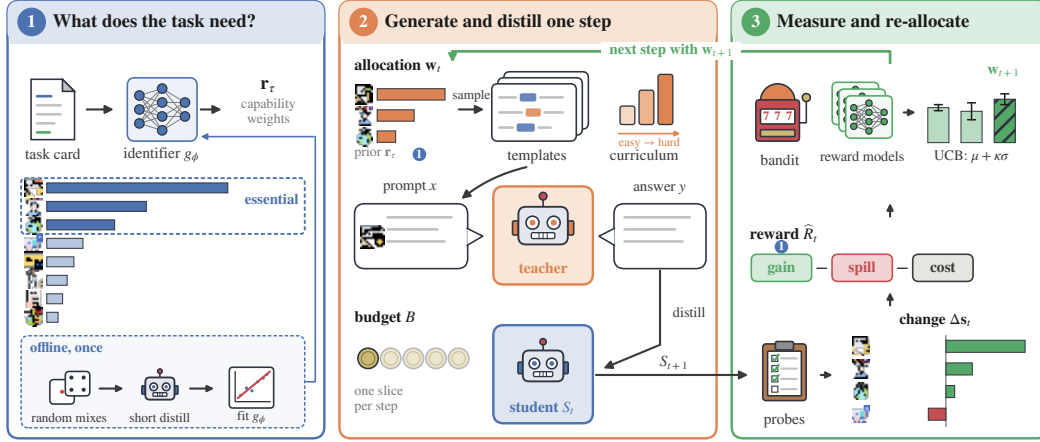


Figure 2: Overview of ReAD. (1) An identifier g_ϕ , pretrained once on low-budget probe interventions, maps a task card to the requirement vector $\mathbf{r}_\tau$, whose top- k entries form $\mathcal{C}_{\text{ess}}$. (2) At step t , capabilities are sampled from the allocation $\mathbf{w}_t$, prompts are instantiated from typed templates under a difficulty curriculum, and the student is distilled on the teacher’s answers. (3) The probe-profile change $\Delta \mathbf{s}_t$ gives the proxy reward, which trains a bootstrap ensemble whose mean μ and uncertainty σ select $\mathbf{w}_{t+1}$ by UCB; the loop repeats until B is spent. Values in the figure are illustrative.

3 METHODOLOGY

ReAD treats Problem 2.3 as state-dependent budget allocation over capabilities (Figure 2). At each step, it estimates task requirements, generates capability-targeted teacher supervision, and uses an uncertainty-aware contextual bandit to choose the next token allocation. As the student profile changes, ReAD can shift budget away from saturated or high-spillover capabilities. **ReAD is off-policy** (Agarwal et al., 2024): the student learns from teacher completions, never from its own samples. The allocator is orthogonal to this choice, so on-policy losses (Gu et al., 2024; Ko et al., 2024) could be plugged in unchanged.

3.1 IDENTIFYING TASK-ESSENTIAL CAPABILITIES

Task requirement vector. For each downstream task τ , ReAD builds a task card $\mathcal{D}_\tau^{\text{spec}}$ from non-test data (a short description, input/output format, evaluation target, and a few exemplars) and estimates from it a requirement vector $\mathbf{r}_\tau \in \Delta_{|C|}$, where $r_{\tau,c}$ is how useful capability c is for the task utility $U_\tau(\cdot)$. $\mathbf{r}_\tau$ acts as an allocation prior: high-mass capabilities are prioritized, and drops on them are treated as harmful spillover. It lives on the simplex because token allocations cannot be negative; negative sensitivities enter through signed capability changes and the spillover penalty of Section 3.3.

Requirement identifier and its supervision. A lightweight identifier g_ϕ (a small Transformer encoder over the task card, a two-layer MLP, and a softmax head) maps the card to $\mathbf{r}_\tau = g_\phi(\mathcal{D}_\tau^{\text{spec}})$. Since $\mathbf{r}_\tau$ is not observed, we supervise it through low-budget interventions in capability space. Writing $U_\tau(S) = F_\tau(\mathbf{s}(S))$ for the measured capability profile $\mathbf{s}(S) \in \mathbb{R}^{|C|}$, a small intervention that moves the profile by $\Delta \mathbf{s}$ changes the utility by

$$\Delta U_\tau = U_\tau(S') - U_\tau(S) \approx \nabla_{\mathbf{s}} F_\tau(\mathbf{s})^\top \Delta \mathbf{s} \quad (3)$$

(first-order expansion, Appendix C), so $\mathbf{r}_\tau$ is trained as a simplex-constrained surrogate for the beneficial part of this local sensitivity, $\mathbf{r}_\tau \approx \Pi_\Delta([\nabla_{\mathbf{s}} F_\tau(\mathbf{s})]_+)$.

Offline pretraining. We build an intervention set $\mathcal{Z} = \{(\mathcal{D}_\tau^{\text{spec}}, \Delta \mathbf{s}_\tau^{(m)}, \Delta U_\tau^{(m)})\}_{\tau,m}$ from auxiliary tasks and non-test splits: for each task we sample sparse allocation vectors $\mathbf{w}^{(m)} \in \Delta_{|C|}$ (a random support with Dirichlet weights), run a short probe distillation with a small budget b_{probe} to obtain an intervened student $S^{(m)}$, and record $\Delta U_\tau^{(m)} = U_\tau(S^{(m)}) - U_\tau(S_0)$ and $\Delta \mathbf{s}_\tau^{(m)} = \mathbf{s}(S^{(m)}) - \mathbf{s}(S_0)$.

The identifier is pretrained to predict the utility change from the capability change,

$$\min_{\phi} \sum_{(\tau, m)} \left(\Delta U_{\tau}^{(m)} - g_{\phi}(\mathcal{D}_{\tau}^{\text{spec}})^{\top} \Delta \mathbf{s}_{\tau}^{(m)} \right)^2 + \lambda_{\text{ent}} \mathcal{H}(g_{\phi}(\mathcal{D}_{\tau}^{\text{spec}})), \quad (4)$$

where the entropy penalty discourages near-uniform requirement vectors. At deployment, $\mathbf{r}_{\tau}$ is computed once per task and reused for the whole run; because the identifier is trained once and shared across tasks, it is not charged to the per-run distillation budget.

3.2 ON-THE-FLY CAPABILITY-TARGETED DATA GENERATION

Given $\mathbf{r}_{\tau}$, ReAD instantiates prompts on the fly from a capability-labeled template library, where each template specifies an instruction scaffold, typed slots filled from deterministic seeds, and output-format constraints. A deterministic difficulty score $d_c(x) \in [0, 1]$ averages capability-specific control factors (e.g., reasoning hops, required functions and tests, or schema rules; Eq. 10 in Appendix C) and splits prompts into easy, medium, and hard tertiles. At step t , ReAD samples capabilities according to $\mathbf{w}_t$, queries the teacher T for a completion y , and immediately distills on (x, y) . A curriculum moves the within-capability mix from easy toward hard examples without affecting the allocation, and per-template caps keep any single scaffold from dominating the generated data.

3.3 CONTEXTUAL BANDIT FOR CAPABILITY ALLOCATION

Decision loop. ReAD runs for $T_{\text{step}} = 20$ decision steps, with the bandit update, data resampling, and proxy refresh at every step (Section 5). At step t , the context summarizes the task requirement, current student profile, remaining budget, and recent allocation history:

$$\mathbf{x}_t = [\mathbf{r}_{\tau}; \mathbf{s}^{\text{probe}}(S_t); b_t; \boldsymbol{\rho}_t], \quad (5)$$

where $\boldsymbol{\rho}_t$ stores a short record of recent allocations and observed gains. The profile $\mathbf{s}^{\text{probe}}(S_t)$ is computed with a small fixed monitoring suite, while full held-out benchmark evaluation is used only for final reporting and coarse calibration.

Candidate allocations and distillation. At each step, ReAD selects an allocation $\mathbf{w}_t$ from a finite candidate set $\mathcal{A}(\tau)$ consisting of local perturbations around the previous allocation and sparse actions concentrated on the top- k capabilities under $\mathbf{r}_{\tau}$. Weights are chosen from a fixed grid and renormalized to sum to one. Given $\mathbf{w}_t$, ReAD samples capability-targeted data and updates the student by minimizing $\mathcal{L}_{\text{distill}}(\theta_t) = -\mathbb{E}_{(x, y)} \sum_{j=1}^{|y|} \log p_{\theta_t}(y_j | x, y_{< j})$ to produce S_{t+1} .

Proxy reward. After the update, ReAD measures the probe-profile change $\Delta \mathbf{s}_t^{\text{probe}} = \mathbf{s}^{\text{probe}}(S_{t+1}) - \mathbf{s}^{\text{probe}}(S_t)$ and forms

$$\hat{R}_t = \mathbf{r}_{\tau}^{\top} \Delta \mathbf{s}_t^{\text{probe}} - \beta \text{Spill}_t - \lambda \text{cost}_t, \quad \text{Spill}_t = \sum_{c \in \mathcal{C}_{\text{ess}}} r_{\tau, c} [-\Delta s_{t, c}^{\text{probe}}]_+ \quad (6)$$

Here $\mathcal{C}_{\text{ess}}$ contains the top- k capabilities under $\mathbf{r}_{\tau}$, and cost_t is the token budget consumed at the step. The first term rewards task-aligned capability gains, the second penalizes regressions on task-essential capabilities, and the third enforces budget awareness.

Reward model and UCB allocation. We append $(\mathbf{x}_t, \mathbf{w}_t, \hat{R}_t)$ to a history buffer and train an ensemble of J two-layer MLP reward regressors $\{h_{\eta_j}\}_{j=1}^J$, each on a bootstrap resample of the 10% profiling split plus the logged transitions (3.2k–4.9k examples per task). For a candidate $\mathbf{w}$, $\mu(\mathbf{x}_t, \mathbf{w})$ and $\sigma(\mathbf{x}_t, \mathbf{w})$ denote the ensemble mean and standard deviation (Appendix C), and ReAD selects the next allocation with an upper-confidence rule,

$$\mathbf{w}_{t+1} = \arg \max_{\mathbf{w} \in \mathcal{A}(\tau)} \mu(\mathbf{x}_{t+1}, \mathbf{w}) + \kappa \sigma(\mathbf{x}_{t+1}, \mathbf{w}), \quad (7)$$

with $b_{t+1} = b_t - \text{cost}_t$. At coarse development checkpoints, we fit an affine calibration from cumulative proxy reward to observed utility change and reduce exploration if the development utility stagnates. Unlike static mixture-regression approaches such as RegMix (Liu et al., 2024), ReAD repeatedly re-estimates the best next allocation from the current student state and remaining budget.

4 THEORETICAL ANALYSIS

We give a local account of the three phenomena ReAD acts on: spillover, diminishing returns, and noisy reward estimates. The assumptions, full statements, and proofs are in Appendix D.

Local cross-capability transfer. Let θ_t be the student parameters after decision step t and $\Delta\theta_t = \theta_t - \theta_{t-1}$. Assume that (i) the expected update under allocation $\mathbf{w}_t$ is a mixture $\sum_c w_{t,c} d_c(\theta_{t-1})$ of capability-conditioned update directions, and (ii) each measured capability is locally a differentiable readout $\tilde{\mathbf{s}}(\theta)$. Then (Proposition D.3)

$$\Delta s_{t,c} \approx \sum_{c' \in \mathcal{C}} w_{t,c'} \Gamma_{c,c'}(\theta_{t-1}), \quad \Gamma_{c,c'}(\theta) := \nabla_{\theta} [\tilde{\mathbf{s}}(\theta)]_c^{\top} d_{c'}(\theta). \quad (8)$$

Γ is the local analogue of the empirical transfer matrix: diagonal terms are intended gains, and an off-diagonal term is nonzero whenever the update direction for c' is not orthogonal to the readout direction of c , which shared representations make the rule rather than the exception. This is why allocating budget to one capability alters others at every decision step.

Budget waste from diminishing returns. Let $b_c \geq 0$ be the tokens assigned to capability c over the run, $\sum_c b_c \leq B$, and suppose task-aligned gains are concave in the budget, $\mathbb{E}[U_{\tau}(S) - U_{\tau}(S_0)] \approx \sum_c r_{\tau,c} G_{\tau,c}(b_c)$ with $G_{\tau,c}$ nondecreasing and concave. Any optimal allocation over a subset $\mathcal{K} \subseteq \mathcal{C}$ equalizes weighted marginal gains, $r_{\tau,c} G'_{\tau,c}(b_c^*) = \nu^*$ for every active c (Proposition D.5); in particular, moving an infinitesimal budget from a single target $\bar{c}$ to another task-relevant capability c' improves the objective whenever $r_{\tau,c'} G'_{\tau,c'}(0) > r_{\tau,\bar{c}} G'_{\tau,\bar{c}}(b_{\bar{c}})$. This is Observation 2 in one line: once the target saturates, further tokens have lower task-aligned marginal value than tokens spent elsewhere. ReAD’s reward in Eq. (6) is richer than this proxy because it also penalizes measured spillover on task-essential capabilities.

Why uncertainty-aware allocation helps. Let $\bar{R}(\mathbf{x}, \mathbf{w}) := \mathbb{E}[\hat{R}_t \mid \mathbf{x}_t = \mathbf{x}, \mathbf{w}_t = \mathbf{w}]$ be the conditional expected proxy reward. If the ensemble is calibrated, $|\mu - \bar{R}| \leq \kappa \sigma$ with high probability, the action chosen by Eq. (7) is within $2\kappa \sigma(\mathbf{x}_t, \mathbf{w}_t)$ of the best action in expected reward (Proposition D.7): the allocator can only be wrong where its own uncertainty is large, which is where the exploration bonus directs budget. This justifies the local ranking rule at each step; it does not claim global optimality for nonconvex fine-tuning.

5 EXPERIMENTAL SETTING

Models and budgets. We mainly distill Llama-3.3-70B-Instruct into Llama-3.1-8B-Instruct, and report Qwen2.5-72B-Instruct to Qwen2.5-14B-Instruct results in Appendix E. All methods use the same student checkpoint, training recipe, and 20M/150M distillation-token budgets.

Evaluation protocol. We evaluate eight capabilities using Appendix Table 3. For each benchmark, 10% of non-test data is used for profiling, requirement inference, and allocation updates; the held-out split is reserved for final reporting. Generation uses temperature 0 with the same prompt template across methods. We report mean $\pm$ standard deviation over three seeds.

Baselines. For each capability, we build a held-out-filtered distillation pool from its benchmark prompts. For multi-dataset capabilities, we use balanced dataset-level sampling. The six main baselines combine three teacher signals—final response (Resp), chain-of-thought plus response (CoT), and token-level logits (Logit)—with two objectives, SFT and KD. Each baseline is one-hot at the capability level, spending the full budget on the target-capability pool. **We also compare against four allocation baselines that share ReAD’s generator, teacher, and budget and differ only in how $\mathbf{w}_t$ is chosen (Table 2):** *Uniform static* ($\mathbf{w}_t = \mathbf{1}/|\mathcal{C}|$), *Task-static mix* ($\mathbf{w}_t = \mathbf{r}_{\tau}$), *Greedy one-step* (re-selects $\mathbf{w}_t$ each step by the ensemble mean alone, $\kappa = 0$), and *Grid-searched* (best fixed allocation in $\mathcal{A}(\tau)$ on the profiling split). These are the matched reference points for adaptive allocation. Finally, we include three published capability-specific distillation methods, adapted to our teacher–student pair and budget: step-by-step distillation (Hsieh et al., 2023) for reasoning, Equation-of-Thought Distillation (EoTD) (Zhu et al., 2024) for math, and CodePLAN (Sun et al., 2024) for code.

ReAD implementation. ReAD runs for 20 decision steps, with bandit updates, proxy refreshes, and data resampling every 1.0M tokens in the 20M setting and every 7.5M tokens in the 150M

Table 1: Capability profile on Llama under 20M- and 150M-token budgets (mean $\pm$ std over three seeds). ReAD achieves the strongest profile at both budgets while using the same student initialization and token budget as all baselines.

Method	General	Steer.	Reason.	Math	Code	Tool	LCU	Multi.
Teacher only	48.13 $\pm$ 0.35	89.98 $\pm$ 0.40	10.51 $\pm$ 0.12	48.34 $\pm$ 0.55	88.40 $\pm$ 0.70	31.90 $\pm$ 0.45	36.20 $\pm$ 0.40	91.10 $\pm$ 0.55
Student only	31.09 $\pm$ 0.40	49.22 $\pm$ 0.55	8.72 $\pm$ 0.15	15.56 $\pm$ 0.60	72.60 $\pm$ 0.95	25.83 $\pm$ 0.60	30.40 $\pm$ 0.45	68.90 $\pm$ 0.80
<i>Budget B = 20M</i>								
Resp-SFT	33.28 $\pm$ 0.47	52.84 $\pm$ 0.71	9.05 $\pm$ 0.18	17.12 $\pm$ 0.83	74.06 $\pm$ 1.09	26.92 $\pm$ 0.78	31.28 $\pm$ 0.51	70.02 $\pm$ 0.93
CoT-SFT	33.02 $\pm$ 0.52	52.16 $\pm$ 0.76	9.46 $\pm$ 0.14	18.58 $\pm$ 0.77	73.82 $\pm$ 1.02	26.81 $\pm$ 0.74	31.19 $\pm$ 0.57	69.78 $\pm$ 0.88
Logit-SFT	33.61 $\pm$ 0.44	53.04 $\pm$ 0.69	9.12 $\pm$ 0.17	17.63 $\pm$ 0.79	74.48 $\pm$ 1.15	27.03 $\pm$ 0.70	31.42 $\pm$ 0.53	70.21 $\pm$ 0.90
Resp-KD	33.83 $\pm$ 0.50	53.62 $\pm$ 0.73	9.18 $\pm$ 0.16	17.84 $\pm$ 0.74	74.23 $\pm$ 1.07	27.18 $\pm$ 0.76	31.58 $\pm$ 0.55	70.48 $\pm$ 0.95
CoT-KD	33.46 $\pm$ 0.46	52.98 $\pm$ 0.78	9.62 $\pm$ 0.13	19.34 $\pm$ 0.72	74.03 $\pm$ 1.10	27.06 $\pm$ 0.72	31.47 $\pm$ 0.58	70.08 $\pm$ 0.89
Logit-KD	34.02 $\pm$ 0.43	54.21 $\pm$ 0.66	9.25 $\pm$ 0.15	18.20 $\pm$ 0.76	74.62 $\pm$ 1.12	27.41 $\pm$ 0.69	31.72 $\pm$ 0.50	70.81 $\pm$ 0.91
ReAD	35.02 $\pm$ 0.41	58.03 $\pm$ 0.61	10.01 $\pm$ 0.12	21.06 $\pm$ 0.68	77.06 $\pm$ 0.97	29.82 $\pm$ 0.64	33.58 $\pm$ 0.49	73.04 $\pm$ 0.84
<i>Budget B = 150M</i>								
Resp-SFT	38.41 $\pm$ 0.58	68.73 $\pm$ 0.94	9.55 $\pm$ 0.14	28.72 $\pm$ 1.08	79.18 $\pm$ 1.26	28.53 $\pm$ 0.82	32.69 $\pm$ 0.73	78.23 $\pm$ 1.12
CoT-SFT	38.02 $\pm$ 0.61	67.92 $\pm$ 0.89	10.14 $\pm$ 0.12	31.58 $\pm$ 0.97	78.86 $\pm$ 1.33	28.41 $\pm$ 0.79	32.83 $\pm$ 0.68	77.77 $\pm$ 1.06
Logit-SFT	39.03 $\pm$ 0.56	69.38 $\pm$ 0.86	9.70 $\pm$ 0.13	29.61 $\pm$ 1.02	79.69 $\pm$ 1.21	28.71 $\pm$ 0.77	33.01 $\pm$ 0.70	78.63 $\pm$ 1.09
Resp-KD	39.62 $\pm$ 0.53	70.98 $\pm$ 0.84	9.82 $\pm$ 0.12	30.21 $\pm$ 0.98	80.09 $\pm$ 1.18	29.09 $\pm$ 0.74	33.42 $\pm$ 0.66	79.18 $\pm$ 1.03
CoT-KD	39.12 $\pm$ 0.55	69.84 $\pm$ 0.88	10.26 $\pm$ 0.11	33.02 $\pm$ 0.92	79.81 $\pm$ 1.24	29.01 $\pm$ 0.71	33.19 $\pm$ 0.69	78.71 $\pm$ 1.01
Logit-KD	40.18 $\pm$ 0.51	72.63 $\pm$ 0.79	9.95 $\pm$ 0.12	31.36 $\pm$ 0.95	80.57 $\pm$ 1.16	29.48 $\pm$ 0.69	33.73 $\pm$ 0.64	79.97 $\pm$ 0.97
ReAD	41.80 $\pm$ 0.45	78.20 $\pm$ 0.70	10.40 $\pm$ 0.10	35.20 $\pm$ 0.80	83.90 $\pm$ 1.00	31.30 $\pm$ 0.55	35.60 $\pm$ 0.55	84.10 $\pm$ 0.90

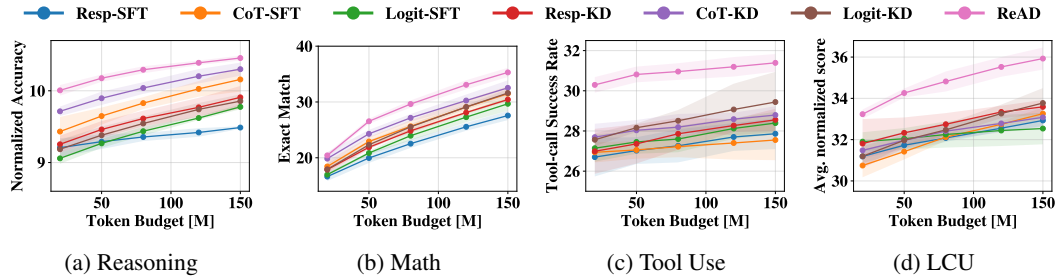


Figure 3: Performance versus token budget for four bottleneck capabilities. Curves show mean $\pm$ standard deviation over three seeds.

setting. Proxy refresh adds 3.12% wall-clock overhead, the full online loop adds 5.63%, and the reusable requirement identifier costs 3.79 GPU-hours to train once. **Cost accounting.** B counts the teacher-generated tokens the student is trained on and is identical for all methods. Two ReAD-specific costs fall outside B and are reported separately rather than folded into it: the identifier is trained once from auxiliary tasks and reused for every task and budget, so its 3.79 GPU-hours are amortized, and the fixed probe suite is the 3.12% overhead above.

6 EXPERIMENTS

We study four questions: **RQ1** utility gains under the same budget, **including against capability-specific distillation methods**; **RQ2** reduced waste and spillover; **RQ3** the value of adaptive bandit scheduling over static or greedy alternatives; and **RQ4** transfer beyond capability-aligned benchmarks. Appendix Figure 5 gives a statistical view of every comparison.

6.1 RQ1: UTILITY UNDER A FIXED BUDGET

Table 1 shows that ReAD achieves the strongest capability profile at 20M tokens and at 150M tokens, outperforming the six one-hot baselines on every capability at both budgets. The gains are especially pronounced on bottleneck capabilities such as Steerability, Math, Tool Use, and LCU. Since each one-hot baseline spends the whole budget on one capability, this isolates the value of distributing the budget at all; the matched comparison for how to distribute it is Table 2(b) (RQ3). The Qwen results in Appendix E show the same ordering, suggesting that the improvement is not tied to a single teacher-student pair. With three seeds, all sixteen differences are positive and all but Reasoning at 150M are significant before correction; after Holm correction over the eight capabilities, Steerability at both budgets and Multilingual at 150M remain significant (Appendix Figure 5). Figure 4b compares

Table 2: **Budget-allocation diagnostics.** (a) 150M-token gain–spillover trade-off from the capability transfer matrix: ReAD keeps near-best on-target gain with much less off-target degradation. (b) Matched-budget allocation schedules under the same generator.

(a) Harmful transfer at 150M				(b) Allocation schedules		
Method	On-tgt $\uparrow$	Off-tgt $\Delta \uparrow$	Neg. $\downarrow$	Method	20M $\uparrow$	150M $\uparrow$
Single-cap. KD	4.79$\pm$0.29	-1.57 $\pm$ 0.21	0.43 $\pm$ 0.03	Uniform static	60.94 $\pm$ 0.48	64.73 $\pm$ 0.36
Task-static mix	4.61 $\pm$ 0.25	-0.87 $\pm$ 0.17	0.29 $\pm$ 0.03	Task-static mix	61.71 $\pm$ 0.43	65.52 $\pm$ 0.33
Greedy one-step	4.72 $\pm$ 0.23	-0.94 $\pm$ 0.19	0.31 $\pm$ 0.03	Greedy one-step	62.03 $\pm$ 0.41	66.01 $\pm$ 0.30
Grid-searched	4.66 $\pm$ 0.24	-0.71 $\pm$ 0.16	0.25 $\pm$ 0.03	Grid-searched	61.88 $\pm$ 0.42	65.81 $\pm$ 0.31
ReAD	4.79 $\pm$ 0.24	-0.38$\pm$0.12	0.17$\pm$0.02	ReAD	63.04$\pm$0.39	67.21$\pm$0.28

ReAD at 150M with the three capability-specific methods, each distilled with its own capability pool under the same evaluation protocol. These methods concentrate their gains on the target capability and transfer weakly elsewhere, producing the imbalanced profiles predicted by the interaction patterns of Section 2.2; ReAD is never worse than them on their own target capability and is substantially better on the non-target ones, giving a stronger and more balanced profile.

6.2 RQ2: BUDGET EFFICIENCY AND SPILLOVER

Figure 3 reports budget-scaling curves for Reasoning, Math, Tool Use, and LCU. ReAD achieves higher performance at the same budget and maintains the strongest scaling trend, indicating that adaptive allocation reduces wasted tokens on low-marginal-return updates. Appendix Table 6 states the same in absolute points: between the two budgets ReAD gains the most per additional 10M tokens on six of the eight capabilities and on average. Conversely, interpolating each baseline log-linearly between its two measured budgets, the strongest one-hot baseline would need 1.3–6.5 $\times$ ReAD’s tokens (median about 2 $\times$) to match ReAD’s 20M score, and none reaches ReAD’s 20M Tool Use score even at 150M (Appendix Table 7). This token saving is an order of magnitude larger than ReAD’s 5.63% online overhead, so the advantage does not rest on the costs that we exclude from the token budget B shared by all methods (Section 5).

Table 2(a) shows that ReAD also improves the gain–spillover trade-off. Single-capability KD achieves strong target gains but causes larger off-target degradation, while static mixing reduces spillover at the cost of weaker target improvement. ReAD better balances the two: it keeps strong on-target gains while suppressing negative transfer across non-target capabilities.

6.3 RQ3: ADAPTIVE SCHEDULING

Table 2(b) compares ReAD with stronger allocation baselines under the same generator and token budgets. Static schedules cannot adapt once marginal returns and spillover change during training, while the greedy heuristic ignores uncertainty and remaining budget. ReAD outperforms both, with a larger margin at 150M where saturation and spillover are stronger. These margins are smaller than those over one-hot baselines (+1.0/+1.2 over Greedy one-step and +1.2/+1.4 over Grid-searched at 20M/150M, with seed standard deviations of 0.3–0.4), as every allocation baseline already mixes capabilities; the remaining gain is the value of re-estimating the mix as the student and the remaining budget change. All four gains remain significant after Holm correction at 150M, and the gain over Uniform static also at 20M (Appendix Figure 5).

Component ablation. We remove one component at a time with everything else fixed: without the requirement identifier the allocation is uniform, without adaptation it is fixed, and without interaction awareness the spillover penalty is dropped. Figure 4a shows that identification and adaptation drive the largest gains, while interaction modeling adds a further improvement by reducing harmful transfer.

6.4 RQ4: TRANSFER BEYOND ALIGNED BENCHMARKS

On held-out XSTest (Röttger et al., 2024), with the same identifier and allocation pipeline and no new capability head, ReAD lowers safe-refusal and raises unsafe-refusal scores over the best SFT/KD baseline at both budgets (Figure 4c; exact values in Appendix Table 8). The four differences agree in direction, but with three seeds none of them survives Holm correction (Appendix Figure 5), so we read this as preliminary evidence of transfer beyond the eight capabilities rather than as a firm result.

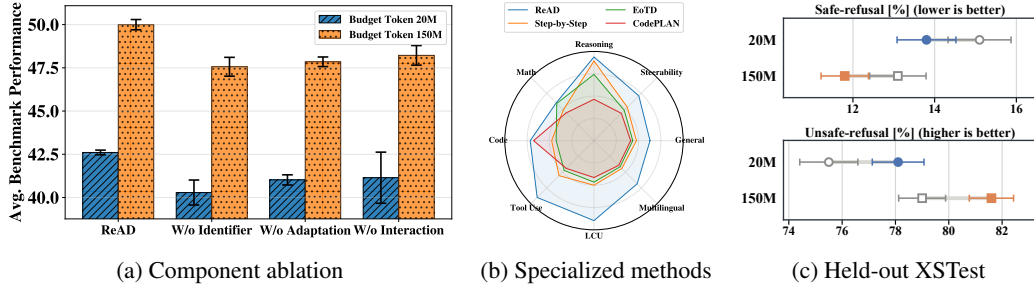


Figure 4: (a) Ablation of ReAD components: average benchmark score over the eight capabilities at 20M and 150M tokens. (b) ReAD versus step-by-step distillation, EoTD, and CodePLAN at 150M; axes show the fraction of the teacher–student gap closed per capability. (c) XSTest scores of ReAD (filled) and the best SFT/KD baseline (hollow), mean $\pm$ std over three seeds; blue circles are 20M and orange squares 150M, as in (a).

7 CONCLUSIONS

We study capability distillation under a fixed token budget and find systematic cross-transfer and diminishing returns, often with increasing harmful spillover on task-critical behaviors. We propose ReAD, a reinforcement-guided framework that identifies task-essential capabilities and adaptively allocates budget across interdependent capabilities via on-the-fly supervision and an uncertainty-aware contextual bandit. Experiments show ReAD improves downstream utility under the same budget while reducing **wasted distillation effort**.

AI USE STATEMENT

In this work, we used generative AI tools to assist with implementing parts of the experimental code (e.g., data processing, evaluation, and plotting scripts) and to improve the writing of the manuscript (grammar, wording, and formatting). We have not used generative AI tools to generate synthetic datasets, develop the theoretical model or conceptual framework, formulate or prove mathematical claims, propose or refine hypotheses, design the research methodology or experiments, or interpret results; the remaining tasks with required disclosure (e.g., translation and qualitative or thematic data analysis) are not applicable to this work. We have reviewed all AI-assisted work: AI-assisted code was checked and tested by the authors before use, and all AI-edited text was verified by the authors against the intended content. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI.

ETHICS STATEMENT

This work does not involve human subjects, crowdsourcing, or personally identifiable data. All experiments use publicly available pretrained models and public benchmarks, and evaluation is fully automated. ReAD is a method for allocating a fixed distillation budget across capabilities; it does not introduce new datasets or release new model weights. As with any distillation method, distilled students may inherit biases or unsafe behaviors from the teacher and the training data, and more efficient distillation may lower the cost of producing specialized models with dual-use capabilities. We discuss these risks and the corresponding mitigations (task-specific safety evaluation, bias and robustness audits, and monitoring of off-target capability changes) in Appendix G.

REPRODUCIBILITY STATEMENT

We have made efforts to ensure that our results are reproducible. Section 5 specifies the teacher–student model pairs, token budgets, evaluation protocol, baselines, and the ReAD update schedule, and Section 3 describes each component of ReAD. Appendix A lists the benchmarks, metrics, and data splits used for each capability, Appendix B details the setting of the exploratory study, and Appendix E reports additional experiments and ablations. The assumptions and full derivations for the theoretical results in Section 4 are given in Appendix D. All reported numbers are means and standard deviations over three seeds. An anonymized repository with our code, configuration files,

and instructions for reproducing the main results is available at <https://anonymous.4open.science/r/ReAD-62B7> (anonymized for review).

REFERENCES

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations (ICLR)*, 2024.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3119–3137, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.172. URL <https://aclanthology.org/2024.acl-long.172>.
- Zinuo Cai, Rongbo Ma, Yicheng Fu, Weishan Zhang, Ruhui Ma, and Haibing Guan. Llmaas: Serving large language models on trusted serverless computing platforms. *IEEE Transactions on Artificial Intelligence*, 2024.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. 2021.
- Daixuan Cheng, Shaohan Huang, and Furu Wei. Adapting large language models via reading comprehension. In *The Twelfth International Conference on Learning Representations*, 2023.
- Wei-Lin Chiang, Zhuohan Li, Ziqing Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. See <https://vicuna.lmsys.org> (accessed 14 April 2023), 2(3):6, 2023.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv*, abs/1803.05457, 2018.
- Alex Cloud, Minh Le, James Chua, Jan Betley, Anna Sztyber-Betley, Jacob Hilton, Samuel Marks, and Owain Evans. Subliminal learning: Language models transmit behavioral traits via hidden signals in data. *arXiv preprint arXiv:2507.14805*, 2025.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.
- Luyang Fang, Xiaowei Yu, Jiazhang Cai, Yongkai Chen, Shushan Wu, Zhengliang Liu, Zhenyuan Yang, Haoran Lu, Xilin Gong, Yufang Liu, Terry Ma, Wei Ruan, Ali Abbasi, Jing Zhang, Tao Wang, Ehsan Latif, Wei Liu, Wei Zhang, Soheil Kolouri, Xiaoming Zhai, Dajiang Zhu, Wenxuan Zhong, Tianming Liu, and Ping Ma. Knowledge distillation and dataset distillation of large language models: Emerging trends, challenges, and future directions. *arXiv preprint arXiv:2504.14772*, 2025.

- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, and Chen-Yu Lee. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. *arXiv preprint arXiv:2305.02301*, 2023.
- Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. Tinybert: Distilling bert for natural language understanding. In *Findings of the association for computational linguistics: EMNLP 2020*, pp. 4163–4174, 2020.
- Yimin Jing, Renren Jin, Jiahao Hu, Huishi Qiu, Xiaohua Wang, Peng Wang, and Deyi Xiong. Followeval: A multi-dimensional benchmark for assessing the instruction-following capability of large language models. *arXiv preprint arXiv:2311.09829*, 2023.
- Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1317–1327, 2016.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Jongwoo Ko, Sungnyun Kim, Tianyi Chen, and Se-Young Yun. DistiLLM: Towards streamlined distillation for large language models. In *International Conference on Machine Learning (ICML)*, 2024.
- Lei Li, Yankai Lin, Shuhuai Ren, Peng Li, Jie Zhou, and Xu Sun. Dynamic knowledge distillation for pre-trained language models. 2021.
- Raymond Li, Erik Nijkamp, Swaroop Mishra, et al. Starcoder: May the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.
- Chen Liang, Simiao Zuo, Qingru Zhang, Pengcheng He, Weizhu Chen, and Tuo Zhao. Less is more: Task-aware layer-wise distillation for language model compression. In *International Conference on Machine Learning*, pp. 20852–20867. PMLR, 2023.
- Qian Liu, Xiaosen Zheng, Niklas Muennighoff, Guangtao Zeng, Longxu Dou, Tianyu Pang, Jing Jiang, and Min Lin. Regmix: Data mixture as regression for language model pre-training. *arXiv preprint arXiv:2407.01492*, 2024.
- Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *arXiv preprint arXiv:2308.08747*, 2023.
- Lucie Charlotte Magister, Jonathan Mallinson, Jakub Adamek, Eric Malmi, and Aliaksei Severyn. Teaching small language models to reason. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 2: short papers)*, pp. 1773–1781, 2023.
- Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. Instruction tuning with gpt-4. *arXiv preprint arXiv:2304.03277*, 2023.
- Edoardo M. Ponti, Goran Glavaš, Olga Majewska, Qianchu Liu, Ivan Vulić, and Anna Korhonen. XCOPA: A multilingual dataset for causal commonsense reasoning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. URL <https://ducdauge.github.io/files/xcopa.pdf>.

- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024.
- Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, 2024.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. Language models are multilingual chain-of-thought reasoners, 2022.
- Kumar Shridhar, Alessandro Stolfo, and Mrinmaya Sachan. Distilling reasoning capabilities into smaller language models. *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 7059–7073, 2023.
- Zhihong Sun, Chen Lyu, Bolun Li, Yao Wan, Hongyu Zhang, Ge Li, and Zhi Jin. Enhancing code generation performance of smaller models by distilling the reasoning ability of llms. *arXiv preprint arXiv:2403.13271*, 2024.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, , and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.
- Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Stanford alpaca: An instruction-following llama model, 2023.
- Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. volume 33, pp. 5776–5788, 2020.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024. URL <https://arxiv.org/abs/2406.01574>.
- Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V. Le, Tengyu Ma, and Adams Wei Yu. DoReMi: Optimizing data mixtures speeds up language model pretraining. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023a.
- Qiantong Xu, Fenglu Hong, Bo Li, Changran Hu, Zhengyu Chen, and Jian Zhang. On the tool manipulation capability of open-source large language models, 2023b.
- Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. A survey on knowledge distillation of large language models. *arXiv preprint arXiv:2402.13116*, 2024.
- Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Berkeley function calling leaderboard. https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html, 2024.
- Chuanpeng Yang, Yao Zhu, Wang Lu, Yidong Wang, Qian Chen, Chenlong Gao, Bingjie Yan, and Yiqiang Chen. Survey on knowledge distillation for large language models: methods, evaluation, and application. *ACM Transactions on Intelligent Systems and Technology*, 2024.

- Yuanhao Yue, Chengyu Wang, Jun Huang, and Peng Wang. Distilling instruction-following abilities of large language models with task-aware curriculum planning. *arXiv preprint arXiv:2405.13448*, 2024.
- Junjie Zhang, Ruobing Xie, Yupeng Hou, Xin Zhao, Leyu Lin, and Ji-Rong Wen. Recommendation as instruction following: A large language model empowered recommendation approach. *ACM Transactions on Information Systems*, 43(5):1–37, 2025.
- Yichi Zhang, Zhuo Chen, Yin Fang, Yanxi Lu, Li Fangming, Wen Zhang, and Huajun Chen. Knowledgeable preference alignment for llms in domain-specific question answering. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 891–904, 2024.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- Qihuang Zhong, Liang Ding, Li Shen, Juhua Liu, Bo Du, and Dacheng Tao. Revisiting knowledge distillation for autoregressive language models. *arXiv preprint arXiv:2402.11890*, 2024.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.
- Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. Distilling mathematical reasoning capabilities into small language models. *Neural Networks*, 179:106594, 2024.

TECHNICAL APPENDICES AND SUPPLEMENTARY MATERIAL

A BENCHMARK DATASETS

Table 3 summarizes the benchmark suite used to evaluate each capability. For capabilities associated with multiple benchmarks, we report the average score across the listed benchmarks.

Table 3: Benchmark suite for evaluating LLM capabilities.

Capability	Benchmarks	Metric
General	MMLU-Pro, BIG	Accuracy
Steerability	IFEval, FollowEval	Instruction-following accuracy
Reasoning	GPQA, ARC-Challenge	Normalized accuracy
Math	MATH Hard, GSM8K	Exact match
Code	HumanEval, MBPP	Pass@1
Tool Use	BFCL V4, ToolBench	Tool-call success rate
LCU	LongBench	Average normalized score
Multilingual	XCOPA, MGSM	Macro accuracy

- **MMLU-Pro** (Wang et al., 2024) tests broad knowledge and problem-solving across many academic subjects using multiple-choice questions.
- **BIG-bench (BIG)** (Suzgun et al., 2022) is a large collection of diverse tasks designed to probe general reasoning and knowledge beyond standard exams.
- **IFEval** (Zhou et al., 2023) measures instruction following by checking whether model outputs satisfy explicit, verifiable constraints in the prompt.
- **FollowEval** (Jing et al., 2023) evaluates how well a model follows user instructions under varied formats and constraint types, complementing IFEval with broader instruction patterns.
- **GPQA** (Rein et al., 2024) is a graduate-level question answering benchmark that stresses hard scientific reasoning with high-quality, expert-written questions.
- **ARC-Challenge** (Clark et al., 2018) contains difficult grade-school science questions that require multi-step reasoning and commonsense beyond keyword matching.
- **MATH Hard** (Hendrycks et al., 2021) is a challenging subset of competition-style math problems and is scored by exact final-answer match.
- **GSM8K** (Cobbe et al., 2021) focuses on grade-school math word problems that require intermediate reasoning steps, also scored by exact final-answer match.
- **HumanEval** (Chen et al., 2021) evaluates code generation via functional correctness on hand-written programming problems, reported as Pass@1.
- **MBPP** (Austin et al., 2021) is a Python programming benchmark with short programming tasks and unit tests, also evaluated by Pass@1.
- **BFCL V4** (Yan et al., 2024) measures tool-use ability by checking whether the model can produce correct function calls under realistic tool specifications.
- **ToolBench** (Xu et al., 2023b) evaluates tool use in more complex, multi-tool scenarios and is scored by tool-call success rate.
- **LongBench** (Bai et al., 2024) evaluates long-context understanding across multiple long-input tasks and reports an average normalized score.
- **XCOPA** (Ponti et al., 2020) tests multilingual causal commonsense reasoning across languages and is reported using macro accuracy.
- **MGSM** (Shi et al., 2022) is a multilingual version of grade-school math and is evaluated by macro accuracy across languages.

B EMPIRICAL STUDY SETTING

We study capability interactions under a fixed distillation budget by distilling a student model toward one target capability at a time and then evaluating the resulting student on *all* capabilities in Table 3. For each capability, we build a capability-specific distillation pool from the corresponding benchmark prompts, excluding any held-out evaluation examples to prevent leakage. We repeat this process under multiple token budgets (e.g., 20M, 80M, 150M tokens) to form a budget-dependent capability transfer matrix, where each entry is the score change on a destination capability when distilling toward a source capability at budget B . We use deterministic decoding for generation benchmarks (temperature = 0) with a fixed prompt template across methods, and we compute each capability score as the average over its associated benchmarks in the table. We instantiate a simple but representative set of capability distillation baselines by combining three supervision forms that expose different levels of teacher knowledge—*Resp* (final answer only), *CoT* (rationale plus final answer), and *Logit* (token-level soft targets)—with two standard objectives, *SFT* and *KD*, yielding six baselines in total.

Scoring and normalization. Each benchmark in Table 3 is scored with its standard metric (accuracy, exact match, pass@1, or tool-call success rate) expressed in percentage points; LongBench reports its own per-task normalized score on the same 0–100 scale. A capability score is the unweighted mean of its benchmark scores, and a transfer-matrix entry $\mathbf{T}_{ij}(B)$ is the absolute difference in capability score between the distilled and the initial student. No min–max rescaling or gap-closure normalization is applied to the reported tables or transfer matrices, so a change of one point is one percentage point of the underlying metric, and small absolute changes remain small in the figures.

C METHOD DETAILS

The following details were moved here from Section 3.

Local utility sensitivity. With $U_\tau(S) = F_\tau(\mathbf{s}(S))$, a small intervention that changes the profile from $\mathbf{s}$ to $\mathbf{s} + \Delta\mathbf{s}$ gives

$$F_\tau(\mathbf{s} + \Delta\mathbf{s}) = F_\tau(\mathbf{s}) + \nabla_{\mathbf{s}} F_\tau(\mathbf{s})^\top \Delta\mathbf{s} + o(\|\Delta\mathbf{s}\|), \quad (9)$$

which yields the local approximation in Eq. (3). This does not assume that the true sensitivity has no negative coordinates; the requirement vector only converts the sensitivity into a nonnegative budget-allocation prior over capabilities.

Difficulty scoring. For prompt x in capability c , we compute a deterministic difficulty score

$$d_c(x) = \frac{1}{|\mathcal{F}_c|} \sum_{f \in \mathcal{F}_c} \frac{f(x) - \min_{x' \in \mathcal{Q}_c} f(x')}{\max_{x' \in \mathcal{Q}_c} f(x') - \min_{x' \in \mathcal{Q}_c} f(x') + \epsilon}, \quad (10)$$

where $\mathcal{F}_c$ is the set of capability-specific control factors and $\mathcal{Q}_c$ is a calibration prompt pool for capability c . For example, reasoning templates use factors such as the number of constraints, reasoning hops, and symbolic complexity; code templates use the number of required functions and tests; steerability templates use the number of rules and schema fields. We split each capability’s calibration pool into easy, medium, and hard buckets by tertiles of $d_c(x)$.

Ensemble mean and uncertainty. For a candidate allocation $\mathbf{w}$, the ensemble of J reward regressors gives

$$\mu(\mathbf{x}_t, \mathbf{w}) = \frac{1}{J} \sum_{j=1}^J h_{\eta_j}(\mathbf{x}_t, \mathbf{w}), \quad \sigma(\mathbf{x}_t, \mathbf{w}) = \sqrt{\frac{1}{J-1} \sum_{j=1}^J (h_{\eta_j}(\mathbf{x}_t, \mathbf{w}) - \mu(\mathbf{x}_t, \mathbf{w}))^2}. \quad (11)$$

D ADDITIONAL THEORETICAL ANALYSIS

This appendix states the assumptions and results summarized in Section 4. The first two parts were moved here from the main text for space.

D.1 LOCAL CROSS-CAPABILITY TRANSFER

Let $\theta_t \in \mathbb{R}^p$ denote the parameters of the student S_t , and let $\Delta\theta_t = \theta_t - \theta_{t-1}$. We analyze one decision step, which is the regime in which ReAD re-estimates its allocation.

Assumption D.1 (Mixture-structured update). At interval t , given an allocation vector $\mathbf{w}_t \in \Delta_{|C|}$, the expected update admits a local mixture approximation $\mathbb{E}[\Delta\theta_t \mid \mathbf{w}_t, \theta_{t-1}] \approx \sum_{c \in C} w_{t,c} d_c(\theta_{t-1})$, where $d_c(\theta)$ is the capability-conditioned local update direction.

Assumption D.2 (Local capability readout). The measured capability profile $\mathbf{s}(S)$ is represented locally as a differentiable function $\tilde{\mathbf{s}}(\theta)$. Near θ_{t-1} ,

$$\Delta s_{t,c} = [\mathbf{s}(S_t)]_c - [\mathbf{s}(S_{t-1})]_c \approx \nabla_{\theta} [\tilde{\mathbf{s}}(\theta_{t-1})]_c^{\top} \Delta\theta_t. \quad (12)$$

Proposition D.3 (Local transfer decomposition). *Under Assumptions D.1 and D.2, the expected change in capability c satisfies Eq. (8), i.e., $\Delta s_{t,c} \approx \sum_{c' \in C} w_{t,c'} \Gamma_{c,c'}(\theta_{t-1})$ with $\Gamma_{c,c'}(\theta) := \nabla_{\theta} [\tilde{\mathbf{s}}(\theta)]_c^{\top} d_{c'}(\theta)$.*

Proof. Substituting the mixture update of Assumption D.1 into the local readout of Assumption D.2 gives

$$\Delta s_{t,c} \approx \nabla_{\theta} [\tilde{\mathbf{s}}(\theta_{t-1})]_c^{\top} \sum_{c'} w_{t,c'} d_{c'}(\theta_{t-1}) = \sum_{c'} w_{t,c'} \Gamma_{c,c'}(\theta_{t-1}).$$

□

Interpretation. Γ is the local analogue of the empirical transfer matrix: diagonal terms are intended gains, and off-diagonal terms are nonzero whenever the update direction for c' is not orthogonal to the readout direction of c , which is expected under shared representations. This is why allocating budget to one capability alters others at the decision-step level.

D.2 BUDGET WASTE FROM DIMINISHING RETURNS

The previous subsection explains spillover. We now isolate diminishing returns with a simpler cumulative-budget proxy. Let $b_c \geq 0$ be the total number of distillation tokens assigned to capability c , with $\sum_c b_c \leq B$, and let $\mathbf{r}_{\tau} \in \Delta_{|C|}$ be the task requirement vector.

Assumption D.4 (Concave task-aligned gains). For task τ and capability c , there is a nondecreasing concave gain function $G_{\tau,c} : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ such that $\mathbb{E}[U_{\tau}(S) - U_{\tau}(S_0)] \approx \sum_{c \in C} r_{\tau,c} G_{\tau,c}(b_c)$ where $\sum_{c \in C} b_c \leq B$.

Consider a restricted distillation strategy that allocates budget only to a subset $\mathcal{K} \subseteq C$:

$$\max_{\{b_c \geq 0\}_{c \in \mathcal{K}}} \sum_{c \in \mathcal{K}} r_{\tau,c} G_{\tau,c}(b_c) \quad \text{s.t.} \quad \sum_{c \in \mathcal{K}} b_c \leq B. \quad (13)$$

Proposition D.5 (Weighted marginal gain criterion). *Any optimum $\{b_c^*\}_{c \in \mathcal{K}}$ of Eq. (13) equalizes weighted marginal gains among active capabilities: there exists $\nu^* \geq 0$ such that*

$$r_{\tau,c} G'_{\tau,c}(b_c^*) \leq \nu^* \quad \forall c \in \mathcal{K}, \quad r_{\tau,c} G'_{\tau,c}(b_c^*) = \nu^* \quad \text{if } b_c^* > 0. \quad (14)$$

Moreover, in the single-capability case targeting $\bar{c}$, shifting an infinitesimal budget from $\bar{c}$ to another task-relevant capability c' improves the proxy objective whenever $r_{\tau,c'} G'_{\tau,c'}(0) > r_{\tau,\bar{c}} G'_{\tau,\bar{c}}(b_{\bar{c}})$.

Proof. The objective is a nonnegative combination of concave functions and the feasible set is convex, so the Karush–Kuhn–Tucker conditions are necessary and sufficient. With ν^* the multiplier of the budget constraint and $\xi_c \geq 0$ those of $b_c \geq 0$, stationarity gives $r_{\tau,c} G'_{\tau,c}(b_c^*) = \nu^* - \xi_c \leq \nu^*$, and complementary slackness ($\xi_c b_c^* = 0$) gives equality whenever $b_c^* > 0$. For the single-target case, moving ϵ tokens from $\bar{c}$ to c' changes the objective by $\epsilon [r_{\tau,c'} G'_{\tau,c'}(0) - r_{\tau,\bar{c}} G'_{\tau,\bar{c}}(b_{\bar{c}})] + o(\epsilon)$, which is positive under the stated condition. □

Implication for ReAD. This formalizes budget waste: once the target saturates, further tokens have lower task-aligned marginal value than tokens assigned elsewhere. ReAD’s reward in Eq. (6) is richer than this proxy because it also penalizes observed spillover, so the allocator optimizes a step-level reward that accounts for both diminishing returns and cross-capability degradation.

Table 4: Capability profile of different methods under a 20M-token budget (Qwen). ReAD consistently outperforms all six **one-hot** baselines **on every capability**.

Method	General	Steerability	Reasoning	Math	Code	Tool Use	LCU	Multilingual
Teacher only	55.27 $\pm$ 0.46	92.36 $\pm$ 0.52	14.28 $\pm$ 0.18	55.74 $\pm$ 0.78	90.12 $\pm$ 0.88	36.08 $\pm$ 0.63	40.23 $\pm$ 0.57	92.34 $\pm$ 0.66
Student only	40.07 $\pm$ 0.61	70.18 $\pm$ 0.74	11.47 $\pm$ 0.22	28.26 $\pm$ 0.95	81.96 $\pm$ 1.10	30.06 $\pm$ 0.82	34.17 $\pm$ 0.71	83.14 $\pm$ 0.93
Resp-SFT	42.18 $\pm$ 0.86	72.92 $\pm$ 0.98	11.62 $\pm$ 0.24	31.42 $\pm$ 1.35	83.88 $\pm$ 1.42	30.96 $\pm$ 0.97	35.04 $\pm$ 0.88	84.36 $\pm$ 1.21
CoT-SFT	41.93 $\pm$ 0.90	72.41 $\pm$ 1.02	12.06 $\pm$ 0.21	33.12 $\pm$ 1.28	83.54 $\pm$ 1.50	30.74 $\pm$ 0.95	34.92 $\pm$ 0.91	84.09 $\pm$ 1.18
Logit-SFT	42.57 $\pm$ 0.82	73.56 $\pm$ 0.95	11.81 $\pm$ 0.23	32.24 $\pm$ 1.22	84.06 $\pm$ 1.37	31.13 $\pm$ 0.93	35.21 $\pm$ 0.86	84.58 $\pm$ 1.16
Resp-KD	42.79 $\pm$ 0.80	74.03 $\pm$ 0.92	11.93 $\pm$ 0.22	32.68 $\pm$ 1.18	84.22 $\pm$ 1.30	31.42 $\pm$ 0.90	35.44 $\pm$ 0.83	84.92 $\pm$ 1.10
CoT-KD	42.44 $\pm$ 0.83	73.37 $\pm$ 0.94	12.31 $\pm$ 0.19	34.08 $\pm$ 1.12	83.91 $\pm$ 1.28	31.02 $\pm$ 0.88	35.29 $\pm$ 0.85	84.76 $\pm$ 1.08
Logit-KD	43.06 $\pm$ 0.78	74.74 $\pm$ 0.88	12.04 $\pm$ 0.21	33.07 $\pm$ 1.10	84.55 $\pm$ 1.22	31.77 $\pm$ 0.86	35.73 $\pm$ 0.80	85.27 $\pm$ 1.02
ReAD	44.63 $\pm$ 0.73	78.57 $\pm$ 0.86	12.86 $\pm$ 0.17	36.24 $\pm$ 1.05	86.38 $\pm$ 1.18	33.41 $\pm$ 0.79	36.98 $\pm$ 0.77	87.23 $\pm$ 0.95

D.3 UNCERTAINTY-AWARE UCB ALLOCATION

Here, we show that action-level regret is controlled by predictive uncertainty. At interval t , ReAD maintains a bandit context

$$\mathbf{x}_t := [\mathbf{r}_\tau; \mathbf{s}(S_t); b_t], \quad (15)$$

where $\mathbf{r}_\tau \in \Delta$ is the task requirement vector, $\mathbf{s}(S_t) \in \mathbb{R}^{|C|}$ is the probe-defined capability profile of the current student, and b_t is the remaining budget. Let $\mathcal{A}(\tau)$ be the discrete candidate action set of allocation vectors. Define the conditional expected proxy reward

$$\bar{R}(\mathbf{x}, \mathbf{w}) := \mathbb{E} \left[\hat{R}_t \mid \mathbf{x}_t = \mathbf{x}, \mathbf{w}_t = \mathbf{w} \right], \quad (16)$$

where $\hat{R}_t$ is the proxy reward used as shown in Eq. (6).

Assumption D.6 (Calibrated uncertainty). For a confidence level $\delta \in (0, 1)$, with probability at least $1 - \delta$, for all encountered $(\mathbf{x}_t, \mathbf{w})$,

$$|\mu(\mathbf{x}_t, \mathbf{w}) - \bar{R}(\mathbf{x}_t, \mathbf{w})| \leq \kappa \sigma(\mathbf{x}_t, \mathbf{w}), \quad (17)$$

where μ and σ are the ensemble mean and standard deviation defined from the reward regressors, and κ is the exploration coefficient of Eq. (7).

Since ReAD selects allocations by the UCB rule in Eq. (7), we can compare its chosen action $\mathbf{w}_t$ to the optimal action $\mathbf{w}_t^*$ under the same context and bound the resulting gap in conditional expected reward using the calibrated uncertainty condition in Assumption D.6, which leads to the following instantaneous regret bound **on the chosen action**.

Proposition D.7 (Instantaneous regret bound). Let $\mathbf{w}_t^* \in \arg \max_{\mathbf{w} \in \mathcal{A}(\tau)} \bar{R}(\mathbf{x}_t, \mathbf{w})$. Under Assumption D.6,

$$\bar{R}(\mathbf{x}_t, \mathbf{w}_t^*) - \bar{R}(\mathbf{x}_t, \mathbf{w}_t) \leq 2\kappa \sigma(\mathbf{x}_t, \mathbf{w}_t), \quad (18)$$

and consequently,

$$\sum_{t=1}^T \left(\bar{R}(\mathbf{x}_t, \mathbf{w}_t^*) - \bar{R}(\mathbf{x}_t, \mathbf{w}_t) \right) \leq 2\kappa \sum_{t=1}^T \sigma(\mathbf{x}_t, \mathbf{w}_t). \quad (19)$$

Proof sketch. By calibration, the true reward of any action is at most its upper confidence score $\mu + \kappa\sigma$, and the selected action maximizes this score; applying the calibration inequality once more to the selected action yields Eq. (18), and summing over steps gives Eq. (19). This result does not imply a global optimum for nonconvex fine-tuning; it justifies the local ranking rule used by ReAD at each budget step rather than a global guarantee. $\square$

E ADDITIONAL EXPERIMENTS

(The 150M Llama table was merged into Table 1 in the main text.)

Additional evidence showing that ReAD improves capability distillation under a fixed budget. We compare ReAD with six capability distillation baselines. Table 1 (main text) reports the Llama results at $B=20M$ and $B=150M$, and Tables 4 and 5 report the corresponding Qwen results. In

Table 5: Capability profile of different methods under a 150M-token budget (Qwen). ReAD consistently outperforms all six baselines.

Method	General	Steerability	Reasoning	Math	Code	Tool Use	LCU	Multilingual
Teacher only	55.27 $\pm$ 0.46	92.36 $\pm$ 0.52	14.28 $\pm$ 0.18	55.74 $\pm$ 0.78	90.12 $\pm$ 0.88	36.08 $\pm$ 0.63	40.23 $\pm$ 0.57	92.34 $\pm$ 0.66
Student only	40.07 $\pm$ 0.61	70.18 $\pm$ 0.74	11.47 $\pm$ 0.22	28.26 $\pm$ 0.95	81.96 $\pm$ 1.10	30.06 $\pm$ 0.82	34.17 $\pm$ 0.71	83.14 $\pm$ 0.93
Resp-SFT	48.02 $\pm$ 0.92	82.07 $\pm$ 1.04	13.18 $\pm$ 0.20	41.26 $\pm$ 1.38	86.92 $\pm$ 1.40	33.29 $\pm$ 0.96	37.42 $\pm$ 0.88	88.07 $\pm$ 1.20
CoT-SFT	48.29 $\pm$ 0.88	82.61 $\pm$ 0.98	13.72 $\pm$ 0.18	44.43 $\pm$ 1.30	86.58 $\pm$ 1.45	33.05 $\pm$ 0.94	37.28 $\pm$ 0.90	87.96 $\pm$ 1.18
Logit-SFT	48.46 $\pm$ 0.86	83.14 $\pm$ 0.95	13.39 $\pm$ 0.19	42.68 $\pm$ 1.26	87.05 $\pm$ 1.32	33.47 $\pm$ 0.92	37.63 $\pm$ 0.86	88.31 $\pm$ 1.12
Resp-KD	48.78 $\pm$ 0.84	83.76 $\pm$ 0.90	13.51 $\pm$ 0.18	43.62 $\pm$ 1.22	87.37 $\pm$ 1.26	33.92 $\pm$ 0.88	38.02 $\pm$ 0.82	88.64 $\pm$ 1.06
CoT-KD	49.05 $\pm$ 0.82	84.28 $\pm$ 0.88	13.87 $\pm$ 0.17	45.17 $\pm$ 1.18	87.11 $\pm$ 1.20	33.66 $\pm$ 0.86	37.91 $\pm$ 0.84	88.53 $\pm$ 1.02
Logit-KD	49.37 $\pm$ 0.80	85.19 $\pm$ 0.84	13.62 $\pm$ 0.18	44.07 $\pm$ 1.16	87.63 $\pm$ 1.18	34.26 $\pm$ 0.84	38.33 $\pm$ 0.78	88.92 $\pm$ 0.98
ReAD	51.53 $\pm$ 0.76	88.04 $\pm$ 0.90	14.03 $\pm$ 0.15	47.06 $\pm$ 1.05	89.04 $\pm$ 1.12	35.52 $\pm$ 0.78	39.47 $\pm$ 0.74	90.54 $\pm$ 0.92

Table 6: Marginal gain from the 20M to the 150M budget, in points per additional 10M tokens, computed from Table 1 as $(s_{150M} - s_{20M})/13$. Bold: largest per column.

Method	General	Steer.	Reason.	Math	Code	Tool	LCU	Multi.	Mean
Resp-SFT	0.39	1.22	0.04	0.89	0.39	0.12	0.11	0.63	0.48
CoT-SFT	0.38	1.21	0.05	1.00	0.39	0.12	0.13	0.61	0.49
Logit-SFT	0.42	1.26	0.04	0.92	0.40	0.13	0.12	0.65	0.49
Resp-KD	0.45	1.34	0.05	0.95	0.45	0.15	0.14	0.67	0.52
CoT-KD	0.44	1.30	0.05	1.05	0.44	0.15	0.13	0.66	0.53
Logit-KD	0.47	1.42	0.05	1.01	0.46	0.16	0.15	0.70	0.55
ReAD	0.52	1.55	0.03	1.09	0.53	0.11	0.16	0.85	0.60

both budgets and for both model families, ReAD achieves the best overall capability profile and outperforms all six baselines on every capability, with the largest improvements appearing on bottleneck capabilities such as Steerability and Math while still maintaining strong gains on Code, Tool Use, and LCU. Scaling the budget from 20M to 150M strengthens all methods, but ReAD benefits more consistently, indicating that its allocation policy converts additional tokens into broader capability improvements rather than concentrating gains on a narrow subset of skills. Across model families, Qwen starts from a stronger student and teacher on most capabilities and also reaches higher absolute performance after distillation, but the relative ordering between methods is unchanged and ReAD remains the top performer, suggesting that ReAD is robust to the choice of teacher–student backbone and that its advantage is not tied to a specific LLM family. Figure 6 visualizes the same Llama numbers as absolute gains over the initial student, next to the best one-hot baseline of each capability, which makes the per-capability margins directly comparable. Table 6 reports how much each method gains per additional 10M tokens between the two budgets, and Table 7 inverts the comparison: it estimates how many tokens the strongest one-hot baseline would need to reach ReAD’s 20M score on each of the eight capabilities.

Statistical view and exact values. Figure 5 shows ReAD’s advantage in every comparison of Section 6 with its confidence interval. Table 8 lists the XSTest scores shown in Figure 4c, and Table 9 lists the differences and Welch t statistics shown in Figure 5, with significance marks after Holm correction within each block and budget.

Where the margin comes from. ReAD’s margin over the strongest one-hot baseline grows with the headroom between teacher and student: across the eight capabilities, the Spearman correlation between the margin and the teacher–student gap is 0.74 at 150M ($p = 0.04$) and 0.43 at 20M (not significant). The margin is largest on Steerability and Multilingual, where the teacher leads by more than 20 points, and smallest on Reasoning, where it leads by only 1.8 points and where the difference is not significant (Table 9). The small Reasoning margin is therefore consistent with limited headroom on GPQA and ARC-Challenge rather than with a failure of the allocation. In relative terms, ReAD’s gain over the initial student is $1.9\times$ that of the strongest one-hot baseline at 20M and $1.3\times$ at 150M on average, i.e., the relative benefit of allocation is largest when the budget is scarce, while the absolute margin grows with budget ($+2.0$ versus $+2.6$ points on average).

(The comparison with capability-specific distillation methods, the XSTest evaluation, and the significance analysis were moved to Section 6.)

Local proxy validation. The proxy validation supports ReAD’s use of local action ranking. Predicted next-step gains align with observed gains, especially for small budget moves, which matches ReAD’s

Table 7: Equivalent budget: tokens the strongest one-hot baseline would need to match ReAD’s 20M-token score on each capability, obtained by interpolating each baseline log-linearly in the budget between its measured 20M and 150M scores (Table 1) and taking the baseline that needs the fewest tokens. Log-linear interpolation is concave in the budget and therefore gives smaller estimates than linear interpolation (last column).

Capability	ReAD @20M	Strongest baseline	@20M	@150M	Tokens needed	Linear interp.
General	35.02	Logit-KD	34.02	40.18	28M (1.4 $\times$)	41M
Steerability	58.03	Logit-KD	54.21	72.63	30M (1.5 $\times$)	47M
Reasoning	10.01	CoT-KD	9.62	10.26	68M (3.4 $\times$)	99M
Math	21.06	CoT-KD	19.34	33.02	26M (1.3 $\times$)	36M
Code	77.06	Logit-KD	74.62	80.57	46M (2.3 $\times$)	73M
Tool Use	29.82	Logit-KD	27.41	29.48	> 150M (not reached)	> 150M
LCU	33.58	Logit-KD	31.72	33.73	129M (6.5 $\times$)	140M
Multilingual	73.04	Logit-KD	70.81	79.97	33M (1.6 $\times$)	52M

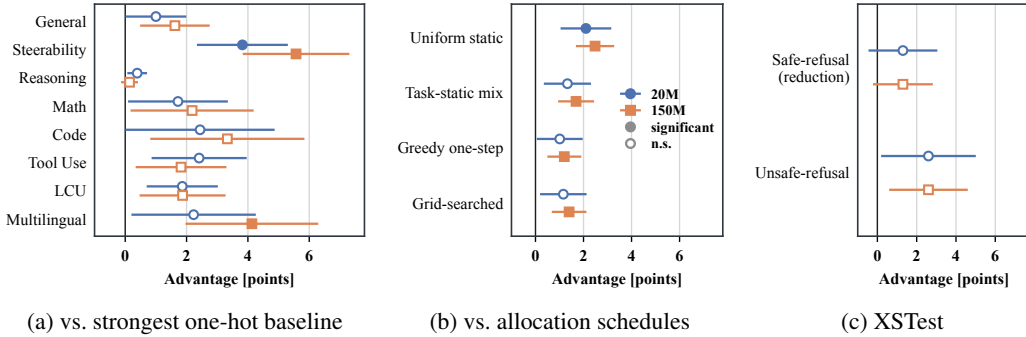


Figure 5: ReAD’s advantage in every reported comparison, with an unadjusted 95% Welch confidence interval computed from the reported means and standard deviations (three seeds); blue circles are 20M and orange squares 150M. Filled markers remain significant ($p < 0.05$) after Holm correction within each panel and budget (across all four tests in (c)); safe-refusal is shown as a reduction, so positive is better everywhere.

stepwise allocation regime. The requirement vectors are stable across runs, and the reward regressors are accurate enough to guide the uncertainty-aware bandit updates. (Moved from the main text.)

F RELATED WORK

Capability Distillation for Large Language Models. Recent work on capability distillation has focused on transferring specific capabilities such as instruction following and context manipulation (Taori et al., 2023; Chiang et al., 2023; Peng et al., 2023; Xu et al., 2023a), thinking patterns (Zhang et al., 2024; Cheng et al., 2023), or text and code generation capability (Zhang et al., 2025; Li et al., 2023). These methods typically steer a teacher LLM via prompts or rationales, generate distillation data or logits, and fine-tune the student on that evidence. However, the majority of methods operate under the assumption that distilling one target capability or domain is sufficient and rarely analyze how that focused training influences broader capability interactions within the student model. To the best of our knowledge, ReAD is the first framework that explicitly accounts for capability interdependence during the distillation.

Distillation objectives, data mixtures, and forgetting. Beyond capability-specific recipes, LLM distillation methods differ in where the training sequences come from. Sequence-level KD and its descendants train on teacher-generated outputs (Hinton et al., 2015; Kim & Rush, 2016), whereas on-policy methods train on student-generated sequences scored by the teacher, which reduces exposure bias (Gu et al., 2024; Agarwal et al., 2024; Ko et al., 2024). ReAD is off-policy in this sense and treats the objective as a plug-in: its contribution is the allocation of a fixed budget across capabilities, not a new loss. A separate line of work optimizes the *mixture* of training data: DoReMi (Xie et al., 2023) and RegMix (Liu et al., 2024) learn static domain weights before, or without observing, the target model’s training trajectory, whereas ReAD re-estimates the mixture online from the student’s

Table 8: Held-out XSTest evaluation. “Best baseline” denotes the best of the six SFT/KD baselines for each metric and budget separately.

Method	20M safe ↓	20M unsafe ↑	150M safe ↓	150M unsafe ↑
Best baseline	15.1 ± 0.77	75.5 ± 1.09	13.1 ± 0.69	79.0 ± 0.88
ReAD	13.8 ± 0.72	78.1 ± 0.97	11.8 ± 0.58	81.6 ± 0.83

Table 9: Welch t -tests (three seeds, unpaired, two-sided) on the means and standard deviations of Tables 1, 2(b), and 8. Δ is ReAD minus the competitor; p values are Holm-corrected within each block and budget, and across all four tests for XSTest (*: $p < 0.05$, **: $p < 0.01$).

Comparison	Competitor	20M		Competitor	150M	
		Δ	t		Δ	t
General	Logit-KD	+1.00	2.9	Logit-KD	+1.62	4.1
Steerability	Logit-KD	+3.82	7.4*	Logit-KD	+5.57	9.1**
Reasoning	CoT-KD	+0.39	3.8	CoT-KD	+0.14	1.6
Math	CoT-KD	+1.72	3.0	CoT-KD	+2.18	3.1
Code	Logit-KD	+2.44	2.9	Logit-KD	+3.33	3.8
Tool Use	Logit-KD	+2.41	4.4	Logit-KD	+1.82	3.6
LCU	Logit-KD	+1.86	4.6	Logit-KD	+1.87	3.8
Multilingual	Logit-KD	+2.23	3.1	Logit-KD	+4.13	5.4*
Avg. utility (Table 2b)	Greedy one-step	+1.01	3.1	Greedy one-step	+1.20	5.1**
Avg. utility (Table 2b)	Grid-searched	+1.16	3.5	Grid-searched	+1.40	5.8**
Avg. utility (Table 2b)	Task-static mix	+1.33	4.0	Task-static mix	+1.69	6.8**
Avg. utility (Table 2b)	Uniform static	+2.10	5.9*	Uniform static	+2.48	9.4**
XSTest safe-refusal ↓	best baseline	−1.3	−2.1	best baseline	−1.3	−2.5
XSTest unsafe-refusal ↑	best baseline	+2.6	3.1	best baseline	+2.6	3.7

measured capability profile and the remaining budget. Finally, the off-target degradation we measure is related to catastrophic forgetting in continual fine-tuning (Kirkpatrick et al., 2017; Luo et al., 2023); ReAD adds no explicit anti-forgetting regularizer but penalizes measured spillover on task-essential capabilities directly in its reward (Eq. 6).

G BROADER IMPACT

ReAD aims to make capability distillation more budget-efficient by allocating limited training tokens across interdependent LLM capabilities. A positive impact is that smaller student models can become more useful under fixed compute budgets, potentially reducing deployment cost and improving access to LLM-based systems in resource-constrained settings. ReAD also explicitly monitors cross-capability transfer, which can help practitioners diagnose when improving one capability degrades other capabilities that a deployment depends on.

At the same time, more efficient distillation may lower the cost of producing specialized models with dual-use capabilities, such as code generation, tool use, or instruction following. Distilled models may also inherit biases, unsafe behaviors, privacy risks, or other limitations from teacher models and training data. In addition, if the capability probes or task requirement vectors are misaligned with the intended deployment objective, ReAD may optimize benchmark utility while overlooking safety-, fairness-, or robustness-relevant behaviors.

We therefore view ReAD as a research framework rather than a deployment-ready safety guarantee. For sensitive applications, practitioners should combine ReAD with task-specific safety evaluation, robustness and bias audits, data filtering, red-team testing, and monitoring of off-target capability changes. Any release or deployment of distilled models should follow the usage restrictions of the underlying models and include appropriate documentation of intended use, limitations, and the safety evaluations that were actually performed.

H LIMITATION AND FUTURE WORK

While ReAD improves budget efficiency by using low-cost capability monitoring and adaptive allocation, it still relies on two practical components that can limit performance. First, the task

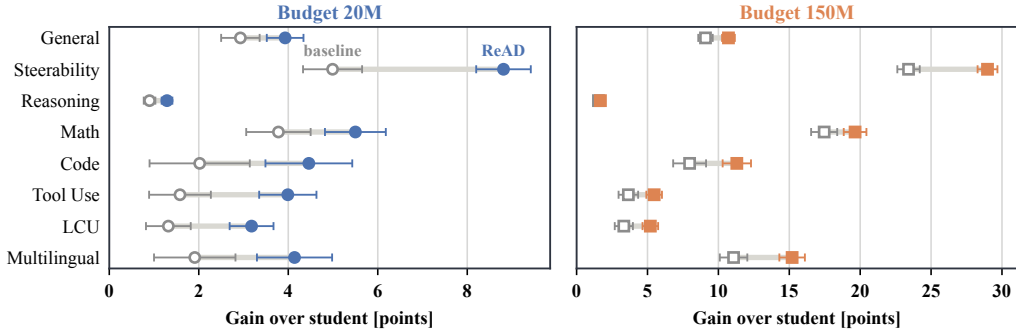


Figure 6: Points gained over the initial student on each capability at 20M (left) and 150M (right) for ReAD and for the best one-hot baseline of that capability, read off Table 1. Error bars are the standard deviation over three seeds of the distilled student’s score. Values are absolute percentage points, so small changes stay small.

requirement estimator and the probe suite may be imperfect: if the inferred essential capabilities or probe signals are misaligned with true task utility, ReAD can allocate budget suboptimally. Second, our capability set and data generator are necessarily finite and template-driven, which may not cover all behaviors a real task depends on, especially for highly domain-specific or long-horizon tasks. Future work can address these limitations by learning more robust and transferable task requirement models, designing probe suites with stronger coverage and better calibration to task utility, and expanding capability definitions and data generation beyond templates (e.g., automatic discovery of capability axes and open-ended curriculum generation). Another promising direction is to jointly optimize allocation and student training dynamics (e.g., optimizer schedules and parameter-efficient adapters) under the same budget, and to extend ReAD to settings with distribution shift, multi-task deployment, or continual updates where requirements and transfer patterns evolve over time.